Airbag ECU Portfolio HS Harz — Functional Safety

AIRBAG ELECTRONIC CONTROL UNIT

Measurement and Processing of Sensor Signals to Control Airbag Operation

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Document Number	SRS-AIRBAG-SW-001
Version	2.1
Date	2026-06-11
Status	DRAFT
Author	Om Naik, Mrunal Ghotekar
Applicable Standards	ISO 26262:2018 / IEC 61508:2010
ASIL / SIL	ASIL D / SIL 3
Target Hardware	Arduino Due (SAM3X8E ARM Cortex-M3) × 2
File	SRS-AIRBAG-SW-001_Rev 2.1.docx
Based On	HARA-AIRBAG-ECU-001 Rev 2.1 | FSRS-AIRBAG-ECU-001 Rev 2.1 | SC-AIRBAG-ECU-001 Rev 2.0 | HADS-AIRBAG-ECU-001 Rev 2.1

Table of Contents

1 Glossary

Table 1: Abbreviations and Definitions

Term / Abbreviation	Definition
ADC	Analog-to-Digital Converter — converts analog voltage 0–3.3 V to 12-bit digital value 0–4095 on Arduino Due
ASIL	Automotive Safety Integrity Level — risk classification per ISO 26262 (QM, A, B, C, D)
CAN	Controller Area Network — serial communication bus
DLC	Data Length Code — CAN frame field indicating payload byte count
DWI	Diagnostic Warning Indicator — red warning LED on pin D10 of each ECU
ECU	Electronic Control Unit
FSR	Functional Safety Requirement — requirement from FSRS-AIRBAG-ECU-001
GPIO	General Purpose Input/Output — digital pin on microcontroller
HADS	Hardware Architecture and Design Specification
HARA	Hazard and Risk Analysis
OMS	OMS Laboratory PC — passive CAN monitoring workstation running USBCANV2
POST	Power-On Self-Test — hardware verification sequence at startup
SIL	Safety Integrity Level per IEC 61508 (1–4)
SRS	Software Requirements Specification — this document
SWR	Software Requirement — individual requirement defined in this document
WDT	Hardware Watchdog Timer — SAM3X8E independent timer; resets MCU on timeout
2oo2	Two-out-of-Two — voting architecture requiring agreement of both channels

2 System Overview

2.1 Hardware Platform

The software runs on two Arduino Due boards, each featuring the Atmel SAM3X8E ARM Cortex-M3 microcontroller running at 84 MHz. Each board independently reads all three sensor inputs, executes the crash detection algorithm, and drives its own AND gate input pins. The hardware DM74LS08N AND gate enforces the physical 2oo2 deployment constraint, both boards must simultaneously assert their output HIGH for the deployment LED to activate.

Both boards transmit on the shared CAN bus via their respective Waveshare SN65HVD230 transceivers. The USB-CAN-A adapter connects to the common CANH/CANL bus and provides the OMS laboratory PC interface for passive monitoring using USBCANV2 software at 500 kbit/s.

2.2 Board Roles and Responsibilities

Both boards are peer-equal. The only difference between ECU 1 and ECU 2 is the three compile-time configuration constants (ECU_ID, DATA_ID, REMOTE_ID). All hardware wiring, pin assignments, and firmware logic are otherwise identical.

Table 2: Board Roles and CAN Identifiers

Define	ECU 1 (Arduino Due 1)	ECU 2 (Arduino Due 2)
ECU_ID	1	2
DATA_ID (TX CAN ID)	0x100	0x200
REMOTE_ID (expected RX ID)	0x200	0x100

Table 3: Software Responsibilities per Board

Responsibility	ECU 1 (Due 1)	ECU 2 (Due 2)
Sensor acquisition (A0/A1/A2)	Yes — reads POTI-1/2/3	Yes — reads same POTI-1/2/3 (shared lines)
Crash detection algorithm	Yes — independent	Yes — independent
Baby seat detection (D11)	Yes — shared signal line	Yes — shared signal line
AND gate drive D8 (Airbag 1)	Yes — pin 1 of DM74LS08N	Yes — pin 2 of DM74LS08N
AND gate drive D9 (Airbag 2)	Yes — pin 4 of DM74LS08N	Yes — pin 5 of DM74LS08N
DWI warning lamp D10	Yes — drives shared LED via 220 Ω	Yes — drives same LED via 220 Ω
CAN TX (own status)	Yes — CAN ID 0x100 on Waveshare 1	Yes — CAN ID 0x200 on Waveshare 2
CAN RX (peer status)	Yes — listens for 0x200	Yes — listens for 0x100
OMS PC monitoring	Passive — both IDs visible on shared bus	Passive — both IDs visible on shared bus
Hardware watchdog (WDT)	Yes — SAM3X8E WDT	Yes — SAM3X8E WDT
POST self-test	Yes	Yes

2.3 Software Safety Classification

Table 4: Software Function Safety Classification

Software Function	ISO 26262	IEC 61508	Governing FSR
POST self-tests, crash detection, deployment logic, watchdog	ASIL D	SIL 3	FSR-03, FSR-07, FSR-17, FSR-22
Sensor acquisition and physical conversion	ASIL D	SIL 3	FSR-01, FSR-02
Baby seat detection and passenger suppression	ASIL D	SIL 3	FSR-09, FSR-10, FSR-11
Diagnostic monitoring and safe-state transitions	ASIL D	SIL 3	FSR-18, FSR-20
CAN bus communication (OMS PC monitoring)	ASIL B	SIL 2	FSR-13, FSR-16
Diagnostic Warning Indicator (DWI) lamp	ASIL A	SIL 1	FSR-19
Serial monitor diagnostic output	ASIL A	SIL 1	FSR-21

3 Software Module Definitions

The following table defines every software module constituting the complete embedded firmware. Each module maps to one or more FSR requirements and is assigned an ASIL classification. All modules run on both ECUs unless otherwise noted.

Table 5: Software Module Definitions

Module	Function	Runs On	Cycle	FSR	ASIL
cpu_test()	Arithmetic self-test of CPU core — verifies 12345×6789 = 83,810,205	Both	Once at boot	FSR-17	ASIL C
ram_test()	Checkerboard write/read pattern 0xAAAA / 0x5555 across 128-word buffer	Both	Once at boot	FSR-17, FSR-22	ASIL D
adc_test()	Verify ADC returns a value in valid range 0–4095	Both	Once at boot	FSR-17	ASIL C

watchdog_setup()	Configure SAM3X8E hardware WDT — reset-on-timeout, 10 ms	Both	Once at boot	FSR-22	ASIL D
watchdog_feed()	Kick hardware WDT — first call in every loop iteration	Both	< 10 ms	FSR-22	ASIL D
updateBabySeat()	Read D11, apply 10 ms debounce, set stable babySeat state	Both	Every loop	FSR-09, 10, 11	ASIL D
toPhys()	Scale 12-bit ADC reading to physical unit (g / deg/s / kPa)	Both	Every loop	FSR-01, 02	ASIL D
Sensor read block	Read A0/A1/A2, convert to g / deg/s / kPa via toPhys()	Both	Every loop	FSR-01, 02	ASIL D
Threshold check block	Compare physical values against THRESH_x constants	Both	Every loop	FSR-03, 04	ASIL D
crashLocal decision	OR of accelEx, gyroEx, pressEx threshold flags	Both	Every loop	FSR-03	ASIL D
driveAirbag1/2 logic	Set D8/D9 HIGH only when crash confirmed, no fault, and peer alive	Both	Every loop	FSR-03, 04, 08	ASIL D
DWI logic	Drive D10 HIGH on any local or remote fault flag	Both	Every loop	FSR-19, 20	ASIL A/D
enterSafeState logic	Force D8/D9 LOW when system_fault TRUE	Both	Every loop	FSR-20	ASIL D
CAN heartbeat TX	Transmit own status frame every 500 ms via CAN bus	Both	500 ms	FSR-07, 08	ASIL D
CAN heartbeat RX	Receive and parse peer status frame from peer ECU	Both	Every loop	FSR-07, 08	ASIL D
CAN timeout check	Set fault_peerChannel if no heartbeat received within COMM_TIMEOUT	Both	Every loop	FSR-15	ASIL D
canSend()	Pack 8-byte CAN frame - ECU1 ID 0x100, ECU2 ID 0x200	Both	100 ms	FSR-13, 16, 16A	ASIL B
Serial monitor output	Print diagnostic summary every 1000 ms to Serial	Both	1000 ms	FSR-21	ASIL A

4 Software Functional Requirements

4.1 Startup and Power-On Self-Test (POST)

Before the system arms itself and enters the main monitoring loop, every safety-critical hardware resource must be verified. Any failure detected during POST results in the system remaining permanently disarmed deployment outputs D8 and D9 are never set HIGH until all POST checks pass.

SWR-01 POST Execution

The software shall execute cpu_test(), ram_test(), and adc_test() sequentially before entering the main loop. cpu_test() shall verify arithmetic correctness by confirming that 12345 × 6789 equals exactly 83,810,205. ram_test() shall write pattern 0xAAAA and then 0x5555 across a 128-word buffer and verify each value. adc_test() shall call analogRead(A0) and verify the result is within valid 12-bit ADC range [0, 4095]. Results shall be stored in fault_cpu, fault_ram, and fault_adc boolean flags respectively.

ASIL / SIL: ASIL C / SIL 2 **FSR Trace:** FSR-17

SWR-02 POST Completion Before Arming

The software shall not set any airbag output pin (D8 or D9) HIGH until all POST tests have completed and their result flags have been evaluated. If any POST flag is TRUE (fault detected), both D8 and D9 shall remain LOW for the entire current ignition cycle.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-17

SWR-03 ADC Resolution Configuration

The software shall call analogReadResolution(12) during setup() to configure the Arduino Due ADC to 12-bit mode. This produces ADC readings in the range 0–4095 corresponding to 0–3.3 V input voltage.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-01

4.2 Sensor Reading

The three potentiometer-based sensor simulations produce analog voltages from 0 V to 3.3 V on ADC pins A0, A1, and A2. The potentiometer supply rail (leg 1) is connected to the 3.3 V rail, constraining wiper output to the safe ADC input range. The software must read these values on every loop iteration, convert them to physical units, and validate their range before using them in the crash detection algorithm.

SWR-04 Sensor Acquisition Cycle

The software shall read analog pins A0 (accelerometer POTI-1), A1 (gyroscope POTI-2), and A2 (pressure sensor POTI-3) on every main loop iteration. No blocking delay() call shall be used in the main loop.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-01, FSR-02

SWR-05 Physical Unit Conversion

Raw ADC readings shall be converted to physical units using the formula: physical = (raw / 4095.0) × MAX_VALUE, where MAX_VALUE is 100.0 g (accelerometer), 360.0 deg/s (gyroscope), 500.0 kPa (pressure sensor). The divisor shall be the floating-point literal 4095.0 to prevent integer division truncation. Physical values shall be stored as float variables.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-01

SWR-06 Threshold Comparison

Physical values shall be compared against: THRESH_ACCEL = 30.0 g, THRESH_GYRO = 120.0 deg/s, THRESH_PRESS = 200.0 kPa. A boolean threshold-exceeded flag shall be set for each sensor independently (accelEx, gyroEx, pressEx). These flags shall be cleared to FALSE at the start of each loop iteration before recomputation.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-03, FSR-04

4.3 Baby Seat Detection and Passenger Suppression

The passenger airbag suppression logic is an ASIL D safety function protecting against airbag deployment into a child seat. A 10 kΩ pull-up resistor connected to the 3.3 V pin of the Arduino Due is fitted on pin D11. The software implements debouncing and fail-safe default behaviour on top of this hardware measure.

SWR-07 Baby Seat Input Pin Configuration

Pin D11 (baby seat switch) shall be configured as INPUT — not INPUT_PULLUP — because an external 10 kΩ pull-up resistor to 3.3 V is already provided on the hardware. Active-LOW logic applies: LOW (0 V) = baby seat present, HIGH (3.3 V) = adult occupant or empty seat.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-09

SWR-08 10 ms Debounce Filter

The updateBabySeat() function shall be called on every main loop iteration. A state transition shall only be accepted after the pin has remained at the new logic level for a minimum of 10 consecutive milliseconds. A timer shall restart on every pin change. Debounce time is defined by compile-time constant DEBOUNCE_MS = 10.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-11

SWR-09 Suppress-on-Fault Default

If the baby seat pin signal becomes absent, intermittent, or produces level transitions faster than the 10 ms debounce window, the software shall default babySeat = TRUE. This suppresses the passenger airbag in the

absence of a verified safe signal, consistent with the suppress-on-fault strategy defined in FSR-10 and SC-AIRBAG-ECU-001.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-10

4.4 Crash Detection Algorithm

The crash detection algorithm fuses the three independent sensor threshold flags into a single crashLocal boolean decision. This decision is used by both boards independently to drive their respective AND gate input pins.

SWR-10 OR Fusion Across Sensors

crashLocal shall be set TRUE if any one of accelEx, gyroEx, or pressEx is TRUE. A crash event is detected if any single sensor exceeds its threshold. This OR fusion strategy ensures that a crash pulse visible on any one axis triggers deployment without requiring simultaneous threshold exceedance across all three sensors.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-03

SWR-11 No Blocking Code in Main Loop

No call to delay() or any other blocking construct shall appear in the main loop. All timing-based tasks (CAN TX, status print) shall use millis() comparison. The loop shall execute as fast as possible — target cycle time < 1 ms per SWR-NF-01.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-02

4.5 Deployment Output Logic

The deployment output logic evaluates the crash decision, fault state, and occupant classification simultaneously on every loop iteration. The result drives D8 and D9, which are inputs to the physical DM74LS08N AND gate. The AND gate output is only HIGH when both boards simultaneously drive their respective pins HIGH. The AND gate outputs (pins 3 and 6) drive the airbag LEDs directly through 220 Ω current-limiting resistors.

SWR-12 AND Gate Input Drive Logic — Airbag 1 (Driver)

Pin D8 (Airbag 1 AND gate input) shall be set HIGH if and only if all of the following conditions are simultaneously TRUE: (1) crashLocal is TRUE, (2) system_fault is FALSE, (3) the peer board is alive (no CAN timeout fault). If any condition is FALSE, D8 shall be driven LOW immediately within the same loop iteration.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-03, FSR-04, FSR-08

SWR-13 Passenger Airbag Suppression — Airbag 2

Pin D9 (Airbag 2 AND gate input) shall satisfy all conditions of SWR-12 and additionally require babySeat to be FALSE. If babySeat is TRUE, D9 shall be held LOW regardless of crash state, fault state, or peer board state. Passenger airbag suppression takes unconditional priority over all other deployment conditions.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-09, FSR-10

SWR-14 Fault Inhibit — Safe State Enforcement

If system_fault is TRUE (composed of: fault_cpu OR fault_ram OR fault_adc OR fault_peerChannel), both D8 and D9 shall immediately be driven LOW in the same loop iteration that the fault is detected. A board detecting any fault forces its AND gate inputs LOW, which collapses the AND gate output LOW regardless of the peer board state.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-20

4.6 Watchdog Supervision

The SAM3X8E hardware watchdog timer operates completely independently of software. Once configured it cannot be disabled by software. Its purpose is to detect software lockup and force a hardware MCU reset, driving all GPIO to their default LOW state before software re-initialises.

SWR-15 Hardware Watchdog Configuration

watchdog_setup() shall configure the SAM3X8E WDT with reset-on-timeout mode (WDT_MR_WDRSTEN) and a timeout value of WDT_MR_WDV(256 × 10), corresponding to approximately 10 ms. The watchdog shall be configured during setup() before POST tests execute. Once configured, the watchdog cannot be disabled by any software instruction.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-22

SWR-16 Watchdog Feed

watchdog_feed() shall be the first function call executed in every iteration of loop(), with no conditional guard. If any loop iteration fails to call watchdog_feed() within the 10 ms timeout window, the hardware will reset the MCU, forcing all GPIO LOW and preventing deployment. An unconditional guard-free placement ensures the watchdog expires during any deadlock or fault condition.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-22

4.7 Inter-Board CAN Communication

Both boards communicate via the shared CAN bus. Each ECU transmits its own status frame on its assigned CAN ID and monitors the peer ECU's frames. A heartbeat timeout mechanism detects total board failure or communication loss and triggers a safe-state transition.

SWR-17 CAN Status Transmission — Both ECUs

Each board shall transmit its status frame via CAN bus every 500 ms using a non-blocking millis() timer. ECU 1 uses CAN ID 0x100 via Waveshare transceiver 1. ECU 2 uses CAN ID 0x200 via Waveshare transceiver 2. Both transceivers connect to the same shared CANH/CANL bus. Heartbeat interval is defined by compile-time constant HEARTBEAT_MS = 500.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-07

SWR-18 CAN Supervision Timeout

Each board shall monitor the CAN bus continuously for incoming status frames from the peer ECU. If no valid frame with the expected REMOTE_ID is received for a period exceeding COMM_TIMEOUT = 2000 ms, fault_peerChannel shall be set TRUE and both D8 and D9 shall be driven LOW via SWR-14. A 2000 ms timeout ensures a silent peer is detected within one diagnostic cycle, well before any crash event window of 20–40 ms elapses.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-07, FSR-15

SWR-19 CAN Status Frame Parsing

Each board shall parse received status frames from the peer ECU and extract: remoteCrash (bit 0 of status byte), babySeat (bit 1 of status byte), remoteFault (bit 2 of status byte). These values shall be stored in peer-state variables and used by the deployment vote logic in SWR-12 and SWR-14.

ASIL / SIL: ASIL D / SIL 3 **FSR Trace:** FSR-07

4.8 CAN Bus Communication — OMS PC Monitoring

Both ECUs transmit status frames to the shared CAN bus, which is accessible to the OMS laboratory PC via the USB-CAN-A adapter running USBCANV2 software. CAN communication is classified ASIL B and carries monitoring data only. Loss of CAN bus communication to the OMS PC has no effect on the ASIL D deployment decision path.

SWR-20 CAN Initialisation

Both ECU 1 and ECU 2 shall initialise CAN communication at 500 kbit/s by calling Can0.begin(CAN_BPS_500K, 255) during setup(). Both boards connect to the shared CANH/CANL bus via their respective Waveshare SN65HVD230 transceivers. The USB-CAN-A adapter on the same bus provides the OMS PC interface.

ASIL / SIL: ASIL B / SIL 2 **FSR Trace:** FSR-13

SWR-21 CAN Frame Structure

Every CAN frame shall use the board's own DATA_ID (0x100 for ECU 1, 0x200 for ECU 2) and DLC = 8 bytes with the following layout: Byte 0 = A0 raw low byte (rawA0 & 0xFF), Byte 1 = A0 raw high byte (rawA0 >> 8), Byte 2 = A1 raw low byte, Byte 3 = A1 raw high byte, Byte 4 = A2 raw low byte, Byte 5 = A2 raw high byte, Byte 6 = status byte (Bit0=crashLocal, Bit1=babySeat, Bit2=systemFault, Bits3–7=reserved 0), Byte 7 = ECU_ID (1 or 2).

ASIL / SIL: ASIL B / SIL 2 **FSR Trace:** FSR-13, FSR-16

SWR-22 CAN Transmission Rate

Each ECU shall transmit one CAN frame every 500 ms using a non-blocking millis() timer. No delay() shall be used. Byte 7 (ECU_ID) is fixed per board and serves as an implicit frame counter identifier. The OMS PC (USBCANV2) must reconstruct 16-bit ADC values as: raw = (byte_high << 8) | byte_low, then convert using the same firmware formula.

ASIL / SIL: ASIL B / SIL 2 **FSR Trace:** FSR-16

4.9 Diagnostic Warning Indicator

The DWI warning lamp is a red LED on pin D10 of each ECU board. Both ECU D10 outputs drive the same shared LED through separate 220 Ω resistors — either ECU driving HIGH illuminates the lamp (hardware OR logic). The LED colour is RED throughout this implementation.

SWR-23 DWI Activation on Local Fault

Pin D10 (DWI red warning lamp) shall be driven HIGH within one main loop cycle of any local fault flag being set TRUE. Local fault flags are: fault_cpu, fault_ram, fault_adc, fault_peerChannel. The DWI shall remain HIGH for the duration of the fault condition.

ASIL / SIL: ASIL A / SIL 1 **FSR Trace:** FSR-19

SWR-24 DWI Activation on Remote Fault

Pin D10 shall also be driven HIGH if the received remoteFault flag from the peer ECU is TRUE, even if all local fault flags are FALSE. This provides redundant warning indication regardless of which board is faulty, consistent with the shared DWI LED hardware architecture.

ASIL / SIL: ASIL A / SIL 1 **FSR Trace:** FSR-19

5 Non-Functional Software Requirements

Table 6: Non-Functional Software Requirements

ID	Requirement	Limit	FSR	ASIL
SWR-NF-01	Main loop cycle time — complete loop() execution from watchdog_feed() to return shall not exceed this limit under any operating condition	< 1 ms	FSR-02	ASIL D
SWR-NF-02	Hardware watchdog timeout — WDT shall trigger MCU reset if loop execution exceeds this interval	10 ms	FSR-22	ASIL D
SWR-NF-03	Baby seat debounce period — D11 must remain stable for this minimum duration before a state change is accepted	10 ms minimum stable	FSR-11	ASIL D
SWR-NF-04	CAN heartbeat transmit interval — each board transmits its status frame to the peer at this interval	500 ms	FSR-07	ASIL D
SWR-NF-05	CAN communication timeout — absence of a valid peer frame for this duration sets the CAN inter-board fault flag	100 ms without valid peer frame	FSR-15	ASIL D
SWR-NF-06	CAN frame transmit interval — each ECU transmits one 8-byte CAN frame at this rate	100 ms	FSR-16	ASIL B
SWR-NF-07	Serial diagnostic print interval — diagnostic status summary printed to Serial at this rate	1000 ms	FSR-21	ASIL A

SWR-NF-08	ADC resolution — the Arduino Due ADC shall be configured to 12-bit mode	12-bit (0–4095)	FSR-01	ASIL D
SWR-NF-09	CAN bus baud rate	500 kbit/s	FSR-13	ASIL B

6 Software Constants and Thresholds

All safety-critical constants shall be defined as compile-time literals using const or #define. They shall not be stored as runtime variables that could be overwritten by software faults. Any change to a value in this table requires a formal change impact assessment and re-verification of the affected test cases.

Table 7: Compile-Time Safety Constants

Constant	Value	Unit	Meaning	FSR
ADC_MAX	4095.0	—	Arduino Due 12-bit ADC maximum count — floating-point to prevent integer division	FSR-01
ACCEL_MAX	100.0	g	Accelerometer full-scale physical range	FSR-01
GYRO_MAX	360.0	°/s	Gyroscope full-scale physical range	FSR-01
PRESS_MAX	500.0	kPa	Pressure sensor full-scale physical range	FSR-01
THRESH_ACCEL	30.0	g	Frontal crash detection threshold	FSR-03
THRESH_GYRO	120.0	°/s	Rollover detection threshold	FSR-03
THRESH_PRESS	200.0	kPa	Side impact detection threshold	FSR-06
DEBOUNCE_MS	10	ms	Baby seat pin debounce time	FSR-11
HEARTBEAT_MS	500	ms	CAN heartbeat transmit interval	FSR-07
COMM_TIMEOUT	100	ms	CAN link fault threshold — peer silent for this duration triggers safe state	FSR-15
CAN_TX_MS	100	ms	CAN frame transmit interval	FSR-16
CAN_BAUD	CAN_BPS_500K	—	CAN bus speed 500 kbit/s	FSR-13

7 Mandatory Coding Constraints

Table 8: Mandatory Coding Constraints

ID	Constraint	Rationale	How to Verify
CC01	Dynamic memory allocation (malloc, new, delete, realloc) is prohibited throughout the entire codebase	Dynamic allocation can fail at runtime in unpredictable ways and can cause memory fragmentation, leading to undefined behaviour in safety-critical code	Code review — grep for malloc, new, delete in all source files
CC02	Recursive function calls are prohibited	Recursion depth is not statically bounded and can cause stack overflow, corrupting safety-critical variables	Code review — verify call graph has no cycles
CC03	All global and static variables shall be explicitly initialised before first use	Uninitialised variables contain undefined values on startup, potentially causing incorrect deployment decisions	Code review + static analysis tool
CC04	No call to delay() anywhere in the main loop	delay() blocks the entire CPU, preventing watchdog feed, sensor reads, and fault detection during the blocked period	Code review — grep for delay() in loop() scope
CC05	All safety-critical thresholds and timing constants shall be compile-time const literals, not runtime variables	Runtime variables can be corrupted by software faults or stack overflow, changing deployment thresholds without detection	Code review — verify all THRESH_x and timing values are const

CC06	Integer arithmetic shall not be used where floating-point precision is required in conversion formulas	Integer division of (raw / 4095) truncates to zero for all valid raw values below 4095, producing incorrect physical unit conversion	Code review — verify toPhys() uses 4095.0 not 4095
CC07	Requirement numbers shall be referenced as comments above the implementing code block	Provides direct traceability from code to requirements, enabling efficient verification and future change impact assessment	Code review — every SWR must appear as a comment in the code
CC08	watchdog_feed() shall always be the first statement in loop(), with no conditional guard	Any conditional guard could prevent the watchdog feed during a fault condition, defeating the safety purpose of the watchdog	Code review — verify watchdog_feed() is unconditional and first

8 Full Requirements Traceability Matrix

Every software requirement traces back to a functional safety requirement (FSR) and forward to an implementing code module and test case. This matrix ensures complete bidirectional traceability as required by ISO 26262-6 and IEC 61508-3.

Table 9: Requirements Traceability Matrix

SWR ID	Description	FSR	ASIL	Code Module	Test Case
SWR-01	POST execution at boot	FSR-17	ASIL C	cpu_test, ram_test, adc_test	TC-17
SWR-02	No output before POST complete	FSR-17	ASIL D	setup() — POST before loop entry	TC-17
SWR-03	ADC 12-bit resolution	FSR-01, 02	ASIL D	analogReadResolution(12) in setup()	TC-01
SWR-04	Sensor read every loop iteration	FSR-01, 02	ASIL D	analogRead A0/A1/A2 every loop	TC-01, TC-02
SWR-05	Physical conversion formula	FSR-01	ASIL D	toPhys() function	TC-01
SWR-06	Threshold comparison	FSR-03, 04	ASIL D	accelEx, gyroEx, pressEx flags	TC-03, TC-04
SWR-07	Baby seat INPUT mode	FSR-09	ASIL D	pinMode(D11, INPUT) in setup()	TC-09
SWR-08	10 ms debounce filter	FSR-11	ASIL D	updateBabySeat() debounce timer	TC-11
SWR-09	Suppress-on-fault default	FSR-10	ASIL D	updateBabySeat() fault branch	TC-10
SWR-10	Sensor OR fusion	FSR-03	ASIL D	crashLocal = accelEx || gyroEx || pressEx	TC-03
SWR-11	No blocking delay in loop	FSR-02	ASIL D	millis() timers throughout	TC-02
SWR-12	D8 AND gate drive logic	FSR-03, 08	ASIL D	driveAirbag1 logic — D8 output	TC-03, TC-08
SWR-13	D9 passenger suppression	FSR-09, 10	ASIL D	driveAirbag2 logic + babySeat gate	TC-09, TC-10
SWR-14	Fault inhibit — both outputs	FSR-20	ASIL D	system_fault → D8/D9 LOW	TC-20
SWR-15	Hardware watchdog configuration	FSR-22	ASIL D	watchdog_setup() — WDV(256×10)	TC-22
SWR-16	Unconditional watchdog service	FSR-22	ASIL D	watchdog_feed() — first in loop()	TC-22
SWR-17	CAN status transmission — both ECUs	FSR-07	ASIL D	canSend() — millis() 500 ms timer	TC-07
SWR-18	CAN supervision timeout	FSR-07, 15	ASIL D	fault_peerChannel timeout detection	TC-15
SWR-19	CAN status frame parsing	FSR-07	ASIL D	parseCAN() — extract remote state	TC-07

SWR-20	CAN0 init 500 kbps	FSR-13	ASIL B	Can0.begin(CAN_BPS_500K, 255)	TC-13
SWR-21	CAN 8-byte frame structure	FSR-13, 16	ASIL B	canSend() frame layout	TC-13, TC-16
SWR-22	CAN TX	FSR-16	ASIL B	millis() non-blocking CAN timer	TC-16
SWR-23	DWI on local fault	FSR-19	ASIL A	D10 HIGH on local fault flag	TC-19
SWR-24	DWI on remote fault	FSR-19	ASIL A	remoteFault → D10 HIGH	TC-19

9 List of Dependent Documents

Table 10: Dependent Documents

No.	Document Name	Document Number	Revision	Date
1	Hazard and Risk Analysis	HARA-AIRBAG-ECU-001	Rev 2.1	2026-05-06
2	Functional Safety Requirements Specification	FSRS-AIRBAG-ECU-001	Rev 2.1	2026-05-11
3	Safety Concept Document	SC-AIRBAG-ECU-001	Rev 2.0	2026-05-12
4	Hardware Architecture and Design Specification	HADS-AIRBAG-ECU-001	Rev 2.1	2026-06-11
5	ISO 26262:2018	—	2018 Edition	—
6	IEC 61508:2010	—	2010 Edition	—

10 Revision History

Table 11: Revision History

Rev	Date	Author	Change Description
1.0	2026-05-31	Om Naik, Mrunal Ghotekar	Initial release — full SRS covering POST, sensor acquisition, crash detection, deployment logic, watchdog, CAN communication, DWI, non-functional requirements, and traceability matrix.
2.0	2026-06-10	Om Naik, Mrunal Ghotekar	Second release — COMM_TIMEOUT updated to 100 ms. Rolling sequence counter defined. Full coding constraints table added.
2.1	2026-06-11	Om Naik, Mrunal Ghotekar	HADS reference updated to 2.1. Pull-up voltage 3.3 V. Direct LED drive confirmed correct for 3 mm red LEDs. Potentiometer leg 1 confirmed on 3.3 V rail. DWI LED colour corrected to RED. CAN architecture clarified — both ECUs transmit on shared bus. Module table updated accordingly.

11 Approval and Sign-Off

This document is at Revision 2.1 DRAFT status. All approvals below must be confirmed before this SRS is used as the baseline for software implementation and V&V execution.

Role	Name	Signature	Date
Software Safety Engineer			
Hardware Safety Engineer			
Functional Safety Engineer			
V&V Lead			
Project Supervisor			